

Informe de conexiones del Arduino

Cableado, protocolo serial y reparto de responsabilidades entre la Raspberry Pi 5 y el Arduino Mega 2560

Equipo	Bruno, Emilia y Majo
Fecha del informe	17 de septiembre de 2026
Controlador	Arduino Mega 2560 R3 — 21 pines en uso
Fuente de verdad	arduino/mexa/mexa.ino (368 líneas)
Último cambio del firmware	edbf50e — 21 de agosto de 2026

Cómo se armó este informe. Todos los pines, umbrales y comandos fueron leídos del código fuente, no de documentación previa. El contraste sacó a la luz ocho contradicciones entre el código y la documentación: la sección 10 dice cuáles eran y cómo se corrigieron.

1. Arquitectura: quién decide y quién reacciona

MEXA tiene **dos cerebros**, y el reparto entre ellos no es casual. La Raspberry Pi 5 corre visión, voz, IA y proyector: es la que *decide*. El Arduino Mega 2560 mueve motores y brazos y lee los sensores de proximidad: es el que *reacciona*. Los une **un solo cable USB** a 9600 baud.

La frontera está trazada sobre un criterio explícito en el firmware: **un reflejo no puede depender de una máquina ocupada**. La Pi puede estar decodificando voz o sintetizando audio justo cuando el visitante da un paso al frente; el Arduino nunca está ocupado. Por eso el freno de emergencia vive en el Arduino. La Pi elige *a qué distancia* quiere quedarse; el Arduino sólo garantiza que MEXA no atropelle a nadie.

Capa	Responsabilidad	Archivo
Firmware	Mover motores y brazos, leer PIR y ultrasónicos, frenar por reflejo	arduino/mexa/mexa.ino
Transporte serial	Dueño único del puerto: abre, escribe, arma líneas de entrada	modulos/conexion_arduino.py
Telemetría	Traduce el texto del Arduino a datos. Lógica pura, sin hardware	modulos/telemetry.py
Motores	API en castellano sobre el protocolo de una letra	modulos/modulo_motores.py
Brazos	Activa y detiene el gesto de hablar	modulos/modulo_brazos.py
Presencia	Consulta el estado de los PIR que reporta el Arduino	modulos/modulo_sensores.py

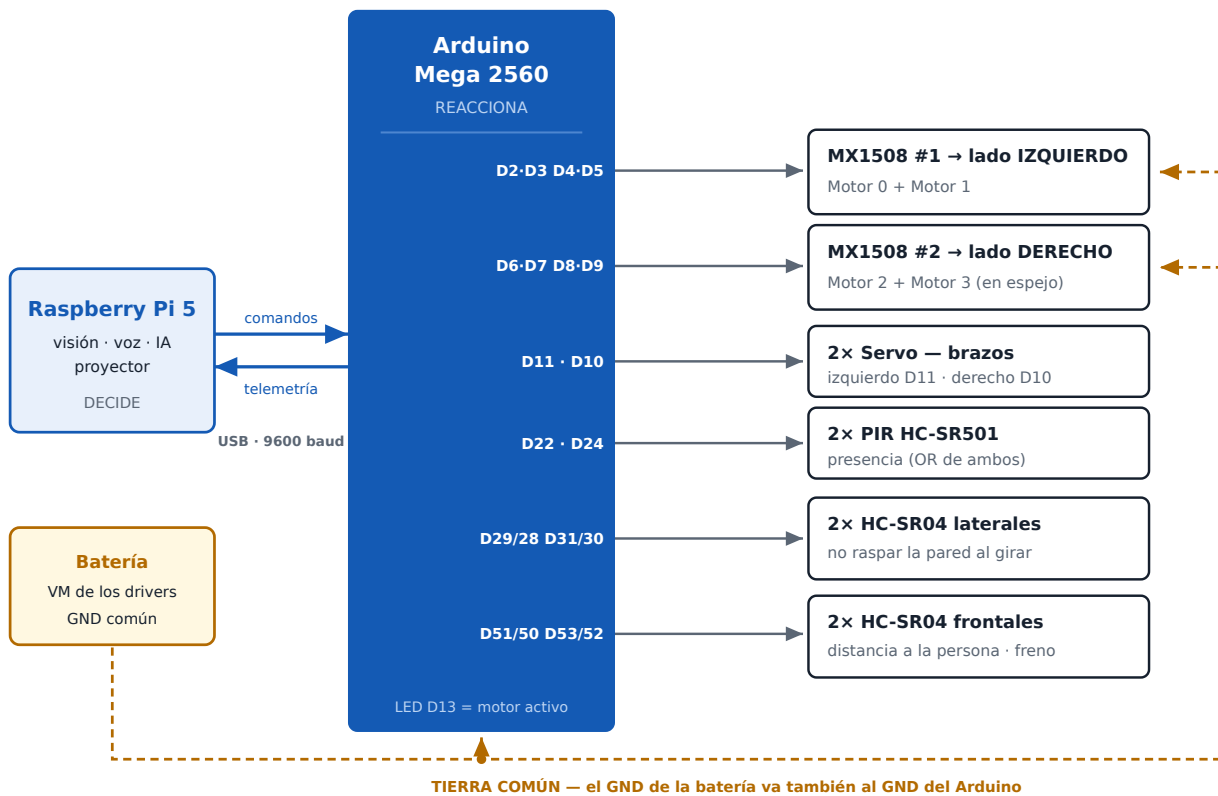
Un solo dueño del puerto. Hay un solo Arduino, así que hay un solo puerto serial. `conexion_arduino.py` es su único propietario: motores, brazos y sensores escriben *a través de él*. Abrir el mismo puerto dos veces daría *device busy* y dejaría a MEXA muda y quieta.

Puerto y velocidad

Parámetro	Valor	Nota
Puerto	/dev/ttyACM0	Mega 2560 R3 original (CDC ACM). Si es un clon con CH340, es /dev/ttyUSB0
Baudrate	9600	≈960 bytes/s en total
Timeout de lectura	1 s	<code>serial.Serial(..., timeout=1)</code>
Espera tras abrir	2 s	El Arduino se reinicia cuando la Pi abre el puerto
Fin de línea	\n	En ambos sentidos. El firmware ignora \n y \r como comandos

Fuente: `modulos/config.py · modulos/conexion_arduino.py:iniciar_conexion()`

2. Diagrama de conexiones



Topología física. El Arduino concentra **todo** lo que se mueve y todo lo que mide proximidad; la cámara, el micrófono y el proyector no pasan por él.

La tierra común no es un detalle. El GND del driver va al negativo de la batería y *también* al GND del Arduino. Sin esa referencia compartida, las señales de dirección que manda el Arduino no tienen contra qué medirse y los motores hacen cualquier cosa — o nada.

3. Tabla maestra de pines

Los 21 pines del Mega que MEXA usa hoy. Es la lista completa: ningún otro pin está declarado en `setup()`.

Pin	Sentido	Componente	Función
D2	SALIDA	MX1508 — Motor 0	Dirección canal A · lado izquierdo
D3	SALIDA	MX1508 — Motor 0	Dirección canal B · lado izquierdo
D4	SALIDA	MX1508 — Motor 1	Dirección canal A · lado izquierdo
D5	SALIDA	MX1508 — Motor 1	Dirección canal B · lado izquierdo
D6	SALIDA	MX1508 — Motor 2	Dirección canal A · lado derecho
D7	SALIDA	MX1508 — Motor 2	Dirección canal B · lado derecho
D8	SALIDA	MX1508 — Motor 3	Dirección canal A · lado derecho
D9	SALIDA	MX1508 — Motor 3	Dirección canal B · lado derecho
D10	SALIDA	Servo brazo derecho	Señal de posición (librería Servo)
D11	SALIDA	Servo brazo izquierdo	Señal de posición (librería Servo)
D13	SALIDA	LED de la placa	Encendido = hay una orden de motor activa
D22	ENTRADA	PIR izquierdo HC-SR501	Movimiento detectado
D24	ENTRADA	PIR derecho HC-SR501	Movimiento detectado
D28	ENTRADA	HC-SR04 lateral derecho	ECHO
D29	SALIDA	HC-SR04 lateral derecho	TRIG
D30	ENTRADA	HC-SR04 lateral izquierdo	ECHO
D31	SALIDA	HC-SR04 lateral izquierdo	TRIG
D50	ENTRADA	HC-SR04 frontal izquierdo	ECHO
D51	SALIDA	HC-SR04 frontal izquierdo	TRIG
D52	ENTRADA	HC-SR04 frontal derecho	ECHO
D53	SALIDA	HC-SR04 frontal derecho	TRIG

Por qué el ECHO va directo, sin divisor de tensión. El HC-SR04 entrega 5 V en su pin ECHO. Un microcontrolador de 3,3 V se dañaría y por eso la receta habitual incluye un divisor resistivo — pero el Mega 2560 es una placa de 5 V y tolera esa tensión en sus entradas. Los cuatro ECHO van directos.

D50-D53 son los pines de SPI del Mega (MISO, MOSI, SCK, SS). Hoy no hay conflicto porque MEXA no tiene ningún periférico SPI. Si mañana se agrega uno — lector de tarjeta SD, módulo de radio —, los ultrasónicos frontales hay que mudarlos a otros pines *antes*, no después.

4. Tren de tracción: 4 motores DC

Cuatro motores de corriente continua, cada uno con dos pines de dirección. Ocho canales en total, así que hacen falta **dos módulos MX1508** (cada uno maneja dos motores).

Motor	Pines	Lado	Invertido	Tecla de diagnóstico
Motor 0	D2 · D3	Izquierdo	no	1
Motor 1	D4 · D5	Izquierdo	no	2
Motor 2	D6 · D7	Derecho	sí	3
Motor 3	D8 · D9	Derecho	sí	4

Los motores derechos se invierten por software, no por cable

Los motores del lado derecho están montados *en espejo* respecto de los izquierdos: la misma tensión los hace girar al revés. En vez de cruzar los cables en el driver — una decisión invisible que muerde meses después — el firmware lo resuelve con una tabla explícita:

```
const bool INVERTIR[4] = { false, false, true, true };
```

Si un motor sale al revés, **se cambia su true/false ahí y nada más**. La corrección queda escrita, versionada y visible, en lugar de escondida en un mazo de cables.

Cómo se traduce cada maniobra

Comando	Maniobra	Lado izquierdo	Lado derecho
F	Adelante	+1	+1
B	Atrás	−1	−1
R	Giro a la derecha	+1	−1
L	Giro a la izquierda	−1	+1
S	Stop	0	0

Los giros son *sobre el eje*: un lado hacia adelante y el otro hacia atrás. MEXA rota en el lugar, sin radio de giro.

No hay PWM. Ninguno. El firmware maneja los pines con `digitalWrite()`: cada motor está *a fondo o parado*, sin estados intermedios. Consecuencia directa: **la velocidad no se puede elegir por software**. Los parámetros `velocidad` de `modulo_motores.py` se aceptan y se *ignoran* — están ahí para no romper a quien ya los llama, y para el día que haya un driver con PWM.

Lo único que sí se elige por software es **cuánto dura** el movimiento: `PULSO_GIRO_S = 1.20`, calibrado en hardware.

5. Brazos: 2 servos con gesto no bloqueante

Parámetro	Valor	Significado
Servo brazo derecho	D10	Señal de posición
Servo brazo izquierdo	D11	Señal de posición
BRAZO_REPOSO	90°	Posición quieta, brazos abajo
BRAZO_MIN / BRAZO_MAX	60° / 120°	Extremos del barrido al hablar
BRAZO_PASO	2°	Grados por actualización
BRAZO_INTERVALO_MS	15 ms	Velocidad del gesto $\approx 133^\circ/\text{s}$

El espejo del brazo derecho

El servo derecho está montado invertido, así que recibe el ángulo reflejado: `brazoDer.write(180 - brazoAng)`. Los dos brazos suben y bajan juntos en vez de ir uno contra el otro. Misma idea que `INVERTIR[]` en los motores: el montaje mecánico se corrige en software, explícito y en un solo lugar.

Por qué el gesto no bloquea

La forma ingenua de animar un servo es un bucle con `delay()`. Sería fatal acá: mientras el Arduino duerme no lee el serial, no vigila los ultrasónicos y no frena. El firmware usa **barrido por millis()**: `actualizarBrazos()` se llama en cada vuelta del `loop()`, mira si pasaron 15 ms, mueve dos grados y *vuelve*. El brazo se mueve y el Arduino sigue atendiendo todo lo demás.

Regla general del firmware: nada que bloquee el `loop()` más de lo indispensable. Es la misma razón por la que los ultrasónicos frontales se miden de a uno por pasada (sección 7) y por la que la distancia se reporta cada 200 ms aunque se mida cada 60.

6. Presencia: 2 sensores PIR

Elemento	Valor	Nota
PIR derecho	D24	Entrada digital
PIR izquierdo	D22	Entrada digital
Lógica	OR	Con que <i>uno</i> detecte movimiento, hay persona
Aviso a la Pi	PRES:0 / PRES:1	Al cambiar de estado y como latido periódico
PRESENCIA_LATIDO_MS	1000 ms	Latido de vida: la Pi sabe que el Arduino sigue vivo

Los PIR **ya no se leen desde la Raspberry Pi**. Antes colgaban del GPIO de la Pi; hoy están en el Arduino y la Pi se entera por serial. Por eso `modulo_sensores.py` no importa `RPi.GPIO`: delega en la conexión compartida.

El latido de un segundo hace dos cosas a la vez: mantiene fresco el estado de presencia aunque no cambie, y le confirma a la Pi que el firmware sigue corriendo. Un silencio de más de un segundo en el canal es, en sí mismo, un síntoma.

7. Ultrasónicos: cuatro HC-SR04, dos trabajos distintos

Los cuatro sensores son el mismo modelo, pero cumplen funciones que no se parecen en nada. **Nunca miden los cuatro a la vez**: el firmware mira al frente cuando avanza y al lateral cuando gira, según lo que MEXA esté haciendo.

7.1 Laterales — no raspar la pared al girar

Sensor	TRIG	ECHO	Se vigila durante
Lateral derecho	D29	D28	Giro a la derecha (R)
Lateral izquierdo	D31	D30	Giro a la izquierda (L)

Si el lateral *del lado hacia el que gira* está a menos de `UMBRAL_PARED_CM = 15 cm`, el Arduino frena los motores y avisa `WALL:D` o `WALL:I`. El giro se cancela.

7.2 Frontales — medir a la persona y frenar

Sensor	TRIG	ECHO	Se vigila durante
Frontal derecho	D53	D52	Avance (F)
Frontal izquierdo	D51	D50	

Mientras MEXA avanza, los frontales informan la distancia a la Pi (`DIST:<izq>,<der>`) y frenan solos si alguien queda a menos de `FRENO_PERSONA_CM = 25 cm`, avisando `STOP:<cm>`.

7.3 Constantes de tiempo y distancia

Constante	Valor	Por qué ese número
UMBRAL_PARED_CM	15 cm	Una pared se esquiva de cerca
FRENO_PERSONA_CM	25 cm	Piso de seguridad, último recurso. <i>Pendiente de calibrar en el robot real</i>
MEDIR_INTERVALO_MS	60 ms	Cada cuánto se mide
REPORTE_INTERVALO_MS	200 ms	Cada cuánto se informa a la Pi
ECHO_TIMEOUT_US	5 000 µs	Laterales: alcance ≈ 85 cm
ECHO_TIMEOUT_FRENTE_US	12 000 µs	Frontales: alcance ≈ 2 m, para ver venir a la persona
SIN_ECO_CM	999	Centinela: no hubo eco. Es un <i>dato</i> , no un error
DISTANCIA_OBJETIVO_CM	70 cm	Dónde quiere pararse MEXA a conversar. Vive en la Pi, calibrado el 21/08/2026

Se mide rápido, se habla lento. El freno es un reflejo y necesita medir seguido (60 ms), pero la Pi sólo necesita saber cuánto falta (200 ms). Reportando cada 60 ms serían ≈272 B/s de los 960 B/s que da 9600 baud: si la Pi se demora un frame de visión, el buffer de salida del Arduino (64 bytes) se llena, `Serial.print` **bloquea** y se congela el `loop()` — con él, el mismísimo reflejo que se quería mejorar.

Y se mide de a uno. `pulseIn` bloquea hasta 12 ms por sensor; los dos frontales juntos congelarían el `loop()` 24 ms por ciclo. Alternando, cada uno se refresca cada ≈120 ms: a la velocidad de MEXA (≈2,5 cm/s), tres décimas de milímetro entre lecturas. El freno, en cambio, se evalúa en **cada** medición y manda el sensor *más cercano*, no el promedio: vale el hombro que entra primero.

8. Protocolo serial

8.1 De la Pi al Arduino — comandos de una letra

Byte	Acción	Detalle
F	Adelante	Activa la vigilancia frontal y olvida las lecturas del avance anterior
B	Atrás	Sin vigilancia: MEXA no tiene sensores traseros
R	Giro a la derecha	Activa la evasión del lateral derecho
L	Giro a la izquierda	Activa la evasión del lateral izquierdo
S	Stop	Frena los cuatro motores
1 ... 4	Mover un solo motor	Diagnóstico: identificar qué motor es cuál
H	Gesticular	Arranca el barrido de brazos (MEXA está hablando)
P	Brazos a reposo	Detiene el gesto y vuelve a 90°

Cualquier otro byte — incluidos `\n` y `\r` — se descarta sin responder.

8.2 Del Arduino a la Pi — líneas terminadas en `\n`

Línea	Significado	Cuándo se emite
MEXA firmware listo: ...	Banner de arranque	Una vez, en <code>setup()</code>
OK <cmd>	Eco del comando	Por cada comando reconocido
PRES:0 / PRES:1	Presencia (PIR)	Al cambiar de estado + latido cada 1 s
DIST:<izq>,<der>	Distancia frontal en cm	Cada 200 ms mientras avanza
STOP:<cm>	Frenó solo: persona cerca	Al cruzar <code>FRENO_PERSONA_CM</code>
WALL:I / WALL:D	Frenó por pared lateral	Al cruzar <code>UMBRAL_PARED_CM</code> girando

8.3 Los reflejos: el Arduino puede terminar un comando antes que la Pi

Ésta es la consecuencia más importante de todo el diseño, y la más fácil de pasar por alto: **un comando puede terminar antes de lo que la Pi pidió**. El firmware frena por su cuenta ante una persona o una pared. Cuando eso pasa, los motores *ya* están parados; la Pi no lo previene, se *entera*.

Función en la Pi	Devuelve	Qué significa
<code>hay_presencia()</code>	<code>bool</code>	Último estado de los PIR
<code>distancia_frontal_cm()</code>	<code>float</code> o <code>None</code>	cm al obstáculo más cercano al frente
<code>freno_por_persona()</code>	<code>float</code> o <code>None</code>	Hecho consumado: el Arduino ya frenó
<code>pared_al_girar()</code>	<code>'I'</code> , <code>'D'</code> o <code>None</code>	Hecho consumado: el Arduino ya canceló el giro

None es «no sé»; 999 es «no hay nadie». Confundirlas es lo que haría que MEXA frene contra un fantasma, o que empuje contra una persona porque el sensor dejó de hablar. El firmware mide al frente *sólo mientras avanza*, así que quieta siempre da `None`. Y si los frontales no están conectados, da `None` siempre y MEXA se guía por la cámara, exactamente como antes. **Ningún sensor ausente puede empeorar lo que ya funcionaba.**

Por qué la Pi debe mandar su propio S cuando se entera de un reflejo. Si el firmware cortó un giro a la décima parte y la Pi no lo registra, el *registro del camino* anota el pulso completo y el regreso gira de más. Un error angular no se suma al final: **rota todos los tramos que vienen después.**

9. Lo que NO pasa por el Arduino

Para que el mapa quede completo: estos periféricos van directo a la Raspberry Pi.

Conector de la Pi	Periférico	Nota
CSI	Cámara Arducam Módulo 3 (IMX708)	Detección de caras con YuNet
USB	Micrófono 360° iTalk-02	Mono de 1 canal: no da dirección de llegada del sonido
USB	Arduino Mega 2560	El canal de este informe
micro-HDMI 1	Proyector KACOTA HY300 Pro	Cara animada y videos

10. Lo que mentía, y qué se hizo

Al contrastar el código con la documentación aparecieron **ocho** desajustes. Ninguno rompía a MEXA en la Pi donde ya está todo instalado, pero cada uno era una trampa para quien volviera a armar el robot desde cero. **Los ocho se corrigieron el 17 de septiembre de 2026**, en la misma sesión que produjo este informe.

El patrón, que importa más que la lista. Todos nacieron igual: el código cambió y la documentación se quedó donde estaba. Documentación desactualizada no es documentación incompleta — es *peor que no tener nada*, porque el lector la sigue creyendo que es verdad y pierde el tiempo buscando una falla eléctrica que no existe. La regla que quedó escrita en el README.txt: si el código cambia un pin, un modelo o un binario, el README se corrige **en el mismo commit**.

Las tres que impedían instalar MEXA desde cero

Qué decía	Qué era verdad	Corrección
pyserial no figuraba en requirements.txt	conexion_arduino.py hace import serial: sin esa librería MEXA no puede moverse	Agregado a requirements.txt
README: ollama pull llama3.2:3b	El código pide llama3.2:1b en tres lugares de modulo_ia.py	Corregido a 1b
README: sudo apt install espeak	modulo_tts.py invoca el binario espeak-ng, que es otro paquete	Corregido a espeak-ng

Las tres juntas dan el mismo resultado: se sigue el README al pie de la letra, y MEXA no arranca. El error aparece recién al ejecutar, nunca al instalar.

Las tres omisiones: pasos obligatorios que no estaban

Faltaba	Sin eso	Corrección
Descargar las voces de Piper a media/tts/	MEXA no tiene voz neural: cae siempre al respaldo	Paso 5, con las URLs verificadas
Descargar los modelos Vosk	MEXA no entiende nada de lo que se le dice	Paso 6
Instalar el AEC de PipeWire	MEXA se oye a sí misma y no se la puede interrumpir	Paso 8

Las dos de código muerto

Dónde	Qué era	Corrección
config.py	IN1..IN4, ENC_IZQ, ENC_DER, FAN_PIN, TEMP_FAN_ON/OFF y cuatro alias de puerto. Nadie los importaba	Borrados. Queda sólo el puerto del Arduino
modulo_proyector.py	El encabezado exigía una carpeta media/imagenes/ con ocho .jpg que el commit a8ec7fb ya había dejado sin uso	Reemplazado por la estructura real de videos

RPi.GPIO también se fue de requirements.txt: MEXA no usa un solo GPIO de la Raspberry, y esa línea hacía fallar pip install -r en cualquier máquina que no fuera una Pi — justo donde se corren los tests de lógica pura.

Dos verdades que la documentación nunca dijo

MEXA no tiene encoders. config.py declaraba ENC_IZQ y ENC_DER, pero el firmware no tiene un solo pin de encoder y nada en la Pi los lee. **No hay odometría:** el regreso al punto de partida se calcula por TIEMPO (PULSO_GIRO_S = 1.20), no por vueltas de rueda. Eso limita la precisión del retroceso y ahora está escrito donde se ve.

FRENO_PERSONA_CM sigue sin calibrar. Es la única de las ocho que *no* se puede cerrar desde el teclado: el firmware la marca ☹ CALIBRAR en el robot real y los 25 cm son un valor de escritorio. Es el último recurso de seguridad frente a una persona — merece una medición con cinta métrica antes de la competencia.

Verificación

Tras la limpieza, los 15 tests de lógica pura del repositorio siguen pasando, y los seis módulos que dependían de config.py importan sin error. La limpieza no tocó una sola línea de comportamiento: sólo se borró lo que no se usaba y se corrigió lo que no era cierto.

11. Cómo verificar el cableado en hardware

El repositorio ya trae las herramientas para comprobar cada bloque. Se corren desde la raíz del proyecto.

Comando	Qué responde
python3 tests/diagnostico_motores.py	¿Dónde se corta la orden? Muestra el banner, manda F y espera OK F
python3 tests/test_manual_arduino.py	Prueba interactiva tecla por tecla: motores, brazos y respuestas del Arduino
python3 tests/probar_ultrasonicos_frontales.py	¿Los frontales miden? ¿Cuánto se desvían de la cinta métrica?
python3 tests/calibrar_pulsos.py giro	Cuánto rota MEXA por pulso, para ajustar PULSO_GIRO_S

Árbol de diagnóstico cuando MEXA «no se mueve y no dice nada»

Síntoma	Dónde está cortada la cadena
No aparece el banner MEXA firmware listo	El comando no llega al firmware: puerto equivocado, sketch viejo o Arduino colgado
Hay banner pero no llega OK F	El firmware no procesa el comando
Llega OK F y el LED D13 enciende	La lógica está 100% sana. El problema es de potencia o cableado de motores , aguas abajo del Arduino: batería, VM del driver o tierra común
Los frontales dicen siempre 999	El sensor no ve nada: revisar TRIG/ECHO y hacia dónde apunta
No llega ni una línea DIST:	El firmware cargado es el viejo, o el avance nunca arrancó

Seguridad antes de probar motores. Elevá el robot con las ruedas en el aire: los scripts mandan F de verdad. Todos mandan S (stop) y P (brazos a reposo) al salir, incluso ante un error o un Ctrl+C.

MEXA · WRO Future Innovators 2026 · Informe generado el 17 de septiembre de 2026. Fuente primaria: `arduino/mexa/mexa.ino`, que es la única autoridad sobre los pines. Si el firmware cambia, este informe se regenera en el mismo commit.